

Security in the Cloud TASK – 2

- 1. Create a custom JSON-based policy in AWS IAM that allows read-only access to all objects in an S3 bucket named 'my-music-playlists', and attach this policy to a new user called 'playlistViewer'.**

Ans :

- Sign in to the AWS Management Console and open the IAM service.
- In the left navigation pane, select Policies and click Create policy.
- Select the JSON tab and paste the following policy:
- {
- "Version": "2012-10-17",
- "Statement": [- {
 - "Sid": "ListBucket",
 - "Effect": "Allow",
 - "Action": "s3:ListBucket",
 - "Resource": "arn:aws:s3:::my-music-playlists"
- },
- {
 - "Sid": "ReadObjects",
 - "Effect": "Allow",
 - "Action": [- "s3:GetObject",
 - "s3:GetObjectVersion"
 -],
 - "Resource": "arn:aws:s3:::my-music-playlists/*"

- }
-]
- }
- Click Next, enter PlaylistViewerS3ReadOnly as the policy name (or another preferred name), optionally add a description, and click Create policy.
- In the left navigation pane, select Users and click Create user.
- Enter playlistViewer as the user name and click Next.
- Choose Attach policies directly.
- Search for and select the custom policy PlaylistViewerS3ReadOnly that you created.
- Click Next, review the user details and attached policy, and click Create user.

**2. Write an inline policy in JSON that allows a user to start and stop EC2 instances only with the tag Environment:Dev, and attach it directly to an existing IAM user named 'devOpsIntern'.

Hint:** Use 'Condition' in your policy to filter by tag.

Ans :

- Sign in to the AWS Management Console and open the IAM service.
- In the left navigation pane, select Users and click the existing user devOpsIntern.
- Open the Permissions tab and click Add permissions.
- Select Create inline policy.
- Choose the JSON tab and paste the following policy:

- {
- "Version": "2012-10-17",
- "Statement": [
- {
 - "Sid": "StartStopDevInstances",
 - "Effect": "Allow",
 - "Action": [
 - "ec2:StartInstances",
 - "ec2:StopInstances"
 -],
 - "Resource": "arn:aws:ec2:*:*:instance/*",
 - "Condition": {
 - "StringEquals": {
 - "ec2:ResourceTag/Environment": "Dev"
 - }
 - }
- }
-]
- }
- Click Next.
- Enter a name for the inline policy, such as StartStopDevInstancesPolicy.
- (Optional) Add a description for the policy.
- Click Create policy (or Create inline policy, depending on the console version).
- Verify that the inline policy is now attached directly to the devOpsIntern IAM user.
- Test the permissions by attempting to start or stop EC2 instances:

- Instances tagged with Environment=Dev can be started and stopped.
- Instances with any other tag value or without the Environment=Dev tag cannot be started or stopped.

3. Create a managed policy in AWS IAM that grants permission to list and view DynamoDB tables, then attach this policy to a group called 'appReviewers' which could represent users reviewing app data like Zomato order history.

Ans :

- Sign in to the AWS Management Console and open the IAM service.
- In the left navigation pane, select Policies and click Create policy.
- Select the JSON tab and paste the following policy:
- {
- "Version": "2012-10-17",
- "Statement": [
- {
 - "Sid": "ListAndViewDynamoDBTables",
 - "Effect": "Allow",
 - "Action": [
 - "dynamodb:ListTables",
 - "dynamodb:DescribeTable",
 - "dynamodb:Scan",
 - "dynamodb:Query",
 - "dynamodb:GetItem",
 - "dynamodb:BatchGetItem"
 -],
 - "Resource": "*"

- }
-]
- }
-

- Click Next.
- Enter DynamoDBReadOnlyPolicy as the policy name (or another preferred name).
- (Optional) Add a description such as Allows users to list and view DynamoDB tables and data.
- Click Create policy.
- In the left navigation pane, select User groups and click the existing group appReviewers.
- Open the Permissions tab and click Add permissions.
- Select Attach policies.
- Search for and select the DynamoDBReadOnlyPolicy managed policy.
- Click Add permissions to attach the policy to the appReviewers group.
- Verify that all users in the appReviewers group can list DynamoDB tables and view table metadata and data, but cannot create, update, or delete tables or items.

**4. Given the following JSON policy, identify and fix the error that would prevent it from granting access as intended:

 { "Version": "2012-10-17", "Statement": [{ "Effect": "Allow", "Action": "s3:GetObject", "Resource": "arn:aws:s3:::user-uploads/*" }] }

Hint: Check if the policy allows listing the bucket as well as getting objects.**

Ans :

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListBucket",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::user-uploads"
    },
    {
      "Sid": "GetObjects",
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::user-uploads/*"
    }
  ]
}
```

- 5. Use ChatGPT to generate a JSON IAM policy that allows users to only update their own profile information in a DynamoDB table named 'UserProfiles', then review and test the policy in the AWS Policy Simulator.

Hint: Ask ChatGPT for a policy with a condition that enforces users can update only their own data.**

Ans :

- {
 - "Version": "2012-10-17",
 - "Statement": [- {
 - "Sid": "UpdateOwnProfileOnly",
 - "Effect": "Allow",
 - "Action": [- "dynamodb:GetItem",
 - "dynamodb:UpdateItem"
 -],
 - "Resource": "arn:aws:dynamodb:*:*:table/UserProfiles",
 - "Condition": {
 - "ForAllValues:StringEquals": {
 - "dynamodb:LeadingKeys": [- "\${aws:username}"
 -]
 - }
 - }
 - }
 -]
 - }
- Attach the policy to the appropriate IAM user, group, or role.
- In the IAM console, open Policy Simulator.
- Select the IAM user, group, or role to which the policy is attached.
- Choose the DynamoDB service.
- Select the GetItem and UpdateItem actions for testing.

- Enter the required simulation parameters (such as the table and partition key value) so that the partition key matches the logged-in user's username.
- Run the simulation and verify that:
 - GetItem and UpdateItem are Allowed when the partition key equals `${aws:username}`.
 - The same actions are Denied when attempting to access another user's profile.